Practicum: PID-regeling van een aeropendulum

Leerdoelen

- ☐ Je kunt uitleggen hoe terugkoppeling de hoek van een aeropendulum regelt.
- ☐ Je kunt de invloed van de P-, I- en D-term experimenteel onderzoeken.
- ☐ Je kunt een regelaar stapsgewijs afstellen en beoordelen op insteltijd, overshoot en stationaire fout.
- ☐ Je kunt meetgegevens uit de Arduino verwerken in Python en de beperkingen van de regeling herkennen.

Veiligheid

1. Gebruik dezelfde gecontroleerde opstelling en voedingsinstellingen als bij het open-loop-practicum. Laat het toegestane hoekbereik en de maximale motoruitgang vaststellen door de practicumbegeleider.
2. Schakel de motorvoeding uit voordat je bedrading, sensor of programma aanpast. De nieuwe CSV-sketch wacht na uploaden of resetten op een expliciet startcommando via Serial Monitor.
3. Houd handen, haar en losse voorwerpen buiten het bewegingsbereik van arm en propeller. Pak de arm niet vast terwijl de motorvoeding is ingeschakeld.
4. Houd de uitschakeling van de motorvoeding bereikbaar. Schakel direct uit bij toenemende oscillaties, een onjuiste hoekmeting of beweging richting een mechanische aanslag. Wacht tot de propeller stilstaat voordat je de opstelling benadert.

Introductie

In het open-looppracticum heb je onderzocht welke hoek ontstaat bij een ingestelde motoruitgang. Die hoek kan veranderen door bijvoorbeeld wrijving of een verandering van de belasting. De besturing merkt zo'n afwijking niet op.

In dit practicum koppelen we de gemeten hoek terug naar de Arduino. De Arduino vergelijkt de hoek met de gewenste waarde en past het PWM-signaal aan. Zo ontstaat een gesloten regelkring. Je onderzoekt eerst een proportionele regelaar, voegt vervolgens een integrerende term toe en onderzoekt ten slotte of een differentiërende term de respons verbetert.

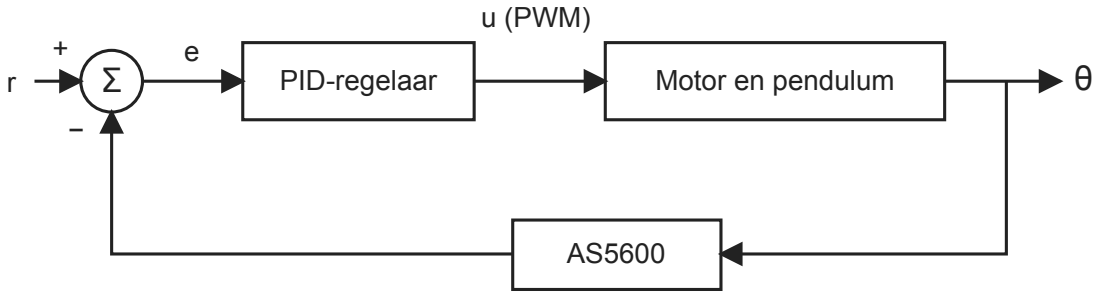
Theorie

Terugkoppeling en regelfout

De gewenste hoek heet het *setpoint*, $r(t)$. De gemeten hoek is $\theta(t)$. De regelfout is

$$e(t) = r(t) - \theta(t). \tag{1}$$

Bij een positieve fout staat de arm onder de gewenste hoek. Voor de gebruikte instelling `DIRECT` moet een grotere motoruitgang leiden tot een grotere gemeten hoek. Controle van dit teken is noodzakelijk voordat je de regeling inschakelt.



Figuur 1: Gesloten regelkring van het aeropendulum. De sensor levert de gemeten hoek voor de terugkoppeling.

De drie bijdragen

Een ideale PID-regelaar kan worden geschreven als

$$u^*(t) = K_p e(t) + K_i \int_0^t e(\tau)\,\mathrm{d}\tau + K_d \frac{\mathrm{d}e}{\mathrm{d}t}. \tag{2}$$

Hier is u^* de berekende uitgang vóór begrenzing. In deze opstelling drukken we de uitgang uit in PWM-telwaarden en de hoek in graden. De eenheden van K_p, K_i en K_d zijn daarom respectievelijk PWM-telwaarden per graad, per graad per seconde en maal seconde per graad.

- **P: reactie op de huidige fout.** Een grotere K_p geeft bij dezelfde fout meer motoruitgang. Dit kan de respons versnellen, maar ook overshoot of oscillaties veroorzaken. Omdat een niet-nul motoruitgang nodig is om de arm omhoog te houden, kan een P-regelaar zonder extra basisuitgang een blijvende fout hebben.
- **I: reactie op de opgebouwde fout.** Een aanhoudende fout bouwt een extra bijdrage op. Hiermee kan de stationaire fout afnemen. Te veel integrerende werking kan leiden tot overshoot en langzaam herstel.
- **D: reactie op snelle verandering.** Deze bijdrage kan de beweging afremmen en overshoot verminderen, maar is gevoelig voor meetruis.

De gebruikte bibliotheek `PID_v1` berekent de D-bijdrage uit de verandering van de *gemeten hoek*: $-K_d\,\mathrm{d}\theta/\mathrm{d}t$. Bij een constant setpoint komt dit overeen met de ideale formule. Bij een setpointsprong wordt zo een directe D-piek door die sprong vermeden. De bibliotheek begrenst ook de opgebouwde integratorbijdrage; langdurige verzadiging kan desondanks de respons beïnvloeden. Zie de broncode van PID_v1 voor de implementatie.

Begrenzing en bemonstering

Voor een maximale PWM-uitgang $u_{\max}$ geldt

$$u(t) = \min\big(\ u_{\max}, \max(0, u^*(t))\ \big), \qquad D \approx \frac{u}{255}. \tag{3}$$

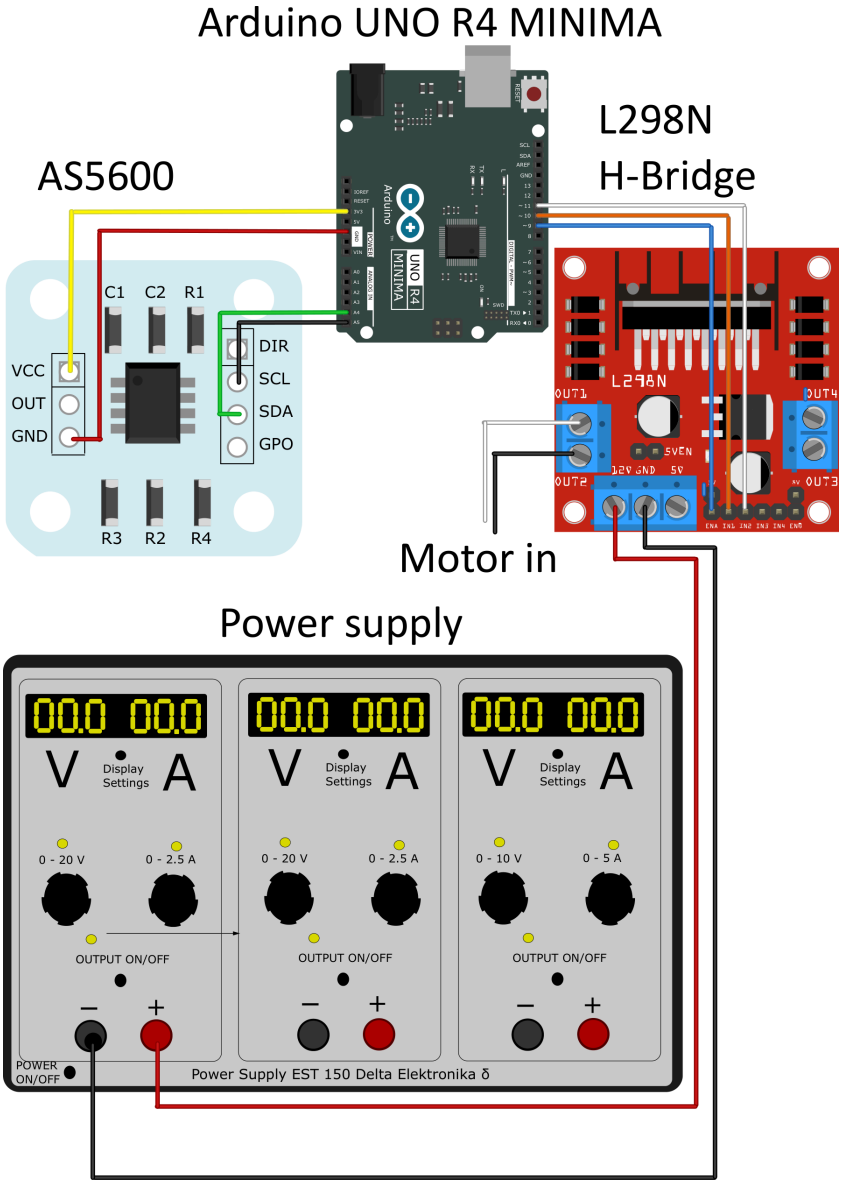
De motoruitgang is een PWM-instelling, geen gemeten spanning of stuwkracht. De regelaar kan in deze sketch geen negatieve stuwkracht vragen. Om de arm te laten zakken, vermindert hij de motoruitgang en werkt de zwaartekracht mee. Daarom hoeven opgaande en neergaande responsen niet gelijk te zijn.

De ingestelde regelperiode is 20 ms, overeenkomend met een nominale frequentie van 50 Hz. Serieel versturen kost echter ook tijd. De nieuwe CSV-sketch verstuurt iedere 20 ms tekst op 115200 baud, in dezelfde ronde als de hoekmeting en regeling. Neem daarom niet zonder controle aan dat opeenvolgende meetpunten precies 20 ms uit elkaar liggen.

Opstelling en Arduino-programma

Gebruik dezelfde Arduino Uno R4 Minima, L298N, AS5600, motor, voeding en mechanische opstelling als bij het open-looppracticum. Houd de voedingsinstellingen tijdens de vergelijking van regelaars gelijk.

Arduino-aansluiting	Component
D9	ENA van de L298N (PWM)
D10 en D11	IN1 en IN2 van de L298N
A4 en A5	SDA en SCL van de AS5600
GND	Gemeenschappelijke massa volgens het aansluitschema



Figuur 2: De aansluitingen blijven gelijk aan die van het open-looppracticum.

Open `Aero_pendulum_PID_CSV.ino` in de gelijknamige submap van `Arduino/PID`. Gebruik de bibliotheken die `Wire.h`, `PID_v1.h` en `AS5600.h` leveren. Bewaar een eigen kopie in een sketchmap met dezelfde naam als het hoofd-`.ino`-bestand. Zet daarin alleen de sketch die je wilt uploaden; meerdere complete sketches met elk een `setup()` en `loop()` horen niet in dezelfde sketchmap.

Instelling	Beginwaarde in de code	Betekenis
`setpoint`	40	Gewenste hoek in graden
`kp`	2	Proportionele versterking
`ki`	0.5	Integrerende versterking
`kd`	0	Differentiërende versterking
`SetSampleTime(20)`	20	Ingestelde regelperiode in ms
`MAX_PWM`	120 (voorlopig)	Grenzen van de motoruitgang

De begininstelling is dus een **PI-regelaar**. Deze waarden zijn geen garantie voor een veilige of goede afstelling van iedere opstelling. Je geeft in Serial Monitor de commando's `S` (start) en `X` (stop). Wijzig het setpoint en de versterkingen in de code en upload opnieuw met uitgeschakelde motorvoeding. Bij iedere start wordt de integrator gewist; tijdens wachten staat de regelaar uit.

De hoek wordt berekend als de ingestelde `ANGLE_OFFSET_DEG` minus de sensorhoek in graden. Dit veronderstelt een bepaalde montage en nulstand van de sensor. De ruwe sensorhoek loopt rond bij een volledige omwenteling. Controleer of de werkhoek correct is en binnen het gekozen bereik geen sprong vertoont.

Voorbereiding

1. Teken de regelkring en benoem setpoint, meetwaarde, regelfout, regelaar, actuator en sensor.
2. Bereken voor $r = 40^\circ$, $\theta = 30^\circ$ en $K_p = 2$ de P-bijdrage. Welke PWM-duty cycle hoort daarbij als de andere bijdragen nul zijn?
3. Gebruik je open-loopmetingen om een haalbare doelhoek en de benodigde stationaire motoruitgang te kiezen. Waarom kan een P-regelaar zonder basisuitgang daarvoor een niet-nul fout nodig hebben?
4. Voorspel wat er verandert wanneer je achtereenvolgens K_p, K_i en K_d verhoogt. Beschrijf ook een mogelijk nadeel per term.
5. Zoek in de sketch de hoekconversie, de uitgangsbegrenzing en het moment waarop de motor wordt aangestuurd. Wat gebeurt er na een reset?
6. Maak een meetplan voor een eerlijke vergelijking van P, PI en PID. Leg beginhoek, doelhoek, meetduur, herhalingen en criteria voor een goede regeling vooraf vast.

Uitvoering

Opdracht 1 — Controle en eerste instelling

1. Controleer met uitgeschakelde motorvoeding de bedrading en bewegingsvrijheid. Lees via USB de sensor uit en beweeg de stilstaande arm voorzichtig. Controleer nulstand, teken en continuïteit van de hoekmeting.
2. Laat een eventuele correctie van de hoekconversie controleren. Een grotere motoruitgang moet binnen het werkgebied een grotere gemeten hoek geven, passend bij `DIRECT`.
3. Kies met de begeleider een haalbaar setpoint uit de open-loopmetingen en een toegestane bovengrens voor PWM. Pas hiervoor `MAX_PWM` aan.
4. Begin met een P-regelaar: zet `ki = 0` en `kd = 0`. Kies een kleine positieve `kp` in overleg met de begeleider. Gebruik niet automatisch de volledige oorspronkelijke PI-instelling.
5. Leg de instellingen en voedingswaarden vast. Upload met uitgeschakelde motorvoeding. Houd bij het inschakelen afstand en controleer of de regeling de afwijking in de juiste richting corrigeert.

Meten vanuit de Arduino IDE

Gebruik de CSV-sketch voor dit practicum in `Arduino/Openloop` of `Arduino/PID`. Open voor open-loopmetingen `Aero_pendulum_OL_CSV.ino` en voor PID-metingen `Aero_pendulum_PID_CSV.ino`, ieder in zijn eigen gelijknamige map. Het programma draait volledig op de Arduino. Python wordt pas achteraf gebruikt voor de analyse.

1. Selecteer in de Arduino IDE de Arduino Uno R4 Minima en de juiste poort. Installeer `AS5600` van Rob Tillaart. Alleen de PID-sketch heeft daarnaast de bibliotheek `PID` van Brett Beauregard nodig.
2. Controleer met de motorvoeding uit de bedrading en hoekconversie. Stel met de begeleider `MAX_PWM`, `MIN_ANGLE_DEG` en `MAX_ANGLE_DEG` vast. De meegeleverde waarden zijn voorlopig.
3. Wijzig voor open-loop `OPEN_LOOP_PWM`; wijzig voor PID `setpoint`, `kp`, `ki` en `kd`. De proefduur staat in `MEASUREMENT_MS` en is standaard 30000 ms. Upload met uitgeschakelde motorvoeding.
4. Open Serial Monitor en kies 115200 baud. Schakel de weergave van extra tijdstempels uit: de sketch levert zelf de tijd. Sluit Serial Plotter en andere programma's op dezelfde poort.
5. Verstuur de hoofdletter `C` voor tien seconden sensorcontrole zonder motoraansturing. Controleer de hoek met alleen USB aangesloten. De uitvoer stopt daarna vanzelf. Een ongeldige sensorstatus of werkhoek levert een foutregel en beëindigt de opname.
6. Maak Serial Monitor leeg vóór de echte meting. Schakel de gecontroleerde motorvoeding in terwijl de sketch wacht. Houd afstand van arm en propeller en verstuur `S`. De Arduino start de motor en verstuurt een CSV-kop gevolgd door meetregels. Een eventuele regeleinde-instelling mag blijven staan; CR en LF worden genegeerd.
7. Na de ingestelde proefduur stopt de Arduino de motor en de uitvoer automatisch. Verstuur `X` om eerder te stoppen. Bij een sensorfout of een hoek buiten het ingestelde bereik stopt de motor eveneens. Schakel na afloop de motorvoeding uit.

Het sluiten van Serial Monitor is geen stopcommando. Gebruik `X` of schakel de motorvoeding uit. Houd de fysieke uitschakeling bereikbaar: de softwarematige stop werkt alleen zolang de microcontroller zijn programma blijft uitvoeren.

CSV-bestand bewaren

De Arduino verstuurt CSV-tekst via USB; zonder opslagmodule schrijft hij geen bestand op de computer. Bewaar de meetgegevens als volgt:

1. Wacht totdat de uitvoer is gestopt. Selecteer en kopieer de volledige opname uit Serial Monitor, vanaf de regel die begint met `t_ms,setpoint_deg` tot en met de laatste volledige meetregel.

2. Plak deze tekst in een leeg tekstbestand, bijvoorbeeld in Kladblok. Sla op als `pid_run1.csv` of `ol_pwm60_run1.csv`. Kies bij opslaan zo nodig “Alle bestanden”, zodat de naam niet eindigt op `.csv.txt`.
3. Gebruik een nieuwe bestandsnaam per proef. Neem geen meldingen van de IDE, geweigerde startmeldingen of een tweede CSV-kop op in hetzelfde bestand. Controleer bij langere proeven of het begin van de uitvoer nog aanwezig is; verkort de proef als de monitor oude regels niet bewaart.

De belangrijkste kolommen zijn `t_ms` (tijd sinds het startcommando), `angle_deg` (hoek), `pwm` (werkelijke PWM-telwaarde), `setpoint_deg` (leeg bij OL), `running` en `sensor_ok`. Noteer de gekozen PWM- of PID-instellingen en de maximale PWM zelf in het labjournaal. Een ongeldige hoek blijft leeg. Het scheidingsteken is een komma en het decimaalteken een punt. Noteer voedingsinstellingen en kalibratie apart in het labjournaal.

Meten, regelen en CSV-uitvoer gebeuren in dezelfde ronde, nominaal iedere 20 ms (50 Hz). Gebruik de tijdstempels voor de tijdas, met $t = 0$ bij het startcommando. De eerste meetregel volgt enkele milliseconden later. Controleer de werkelijke intervallen door opeenvolgende tijdstempels van elkaar af te trekken. Er is geen nulmeting vóór de motoraansturing; bepaal de beginhoek vooraf met `C`. Verdere instructies staan in `Arduino/README.md`.

Opdracht 2 — P-regeling

1. Onderzoek minimaal drie stapsgewijs gekozen waarden van K_p, met $K_i = K_d = 0$. Verhoog niet verder bij toenemende oscillaties of overschrijding van het afgesproken bereik.
2. Gebruik per meting dezelfde ruststand en hetzelfde setpoint. Schakel de motorvoeding uit, wacht tot de arm stilstaat en upload of reset. Schakel de gecontroleerde motorvoeding in terwijl de sketch wacht. Verstuur `S` in Serial Monitor. De Arduino-tijd in de CSV begint bij dat startcommando.
3. Meet tot een stationaire toestand zichtbaar is, of tot de vooraf afgesproken maximale meetduur. Registreer ook wanneer de arm het setpoint niet bereikt.
4. Herhaal iedere instelling minimaal drie keer. Bewaar per proef de CSV en noteer K_p, K_i, K_d, setpoint, PWM-grens en voedingsinstellingen in je labjournaal.
5. Vergelijk snelheid, oscillaties en blijvende fout. Kies een bruikbare P-instelling als uitgangspunt voor de volgende opdracht.

Opdracht 3 — PI-regeling

1. Houd de gekozen K_p gelijk en laat $K_d = 0$. Voeg een kleine positieve K_i toe. Onderzoek minimaal drie waarden en herhaal het meetprotocol van opdracht 2.
2. Start voor iedere proef een nieuwe registratie. Het startcommando wist de integrator en activeert de regeling direct. Daardoor neemt de integrator de geschiedenis van de vorige proef niet mee.
3. Onderzoek of de stationaire fout kleiner wordt en welke invloed K_i heeft op overshoot en insteltijd. Bekijk daarbij ook of de PWM-uitgang tegen een grens blijft staan.
4. Selecteer een PI-instelling en onderbouw de keuze met meetresultaten. Gebruik geen onhaalbaar setpoint om de integrator bewust te laten oplopen.

Opdracht 4 — PID-regeling

1. Houd de gekozen K_p en K_i gelijk. Vergelijk $K_d = 0$ met minimaal twee kleine positieve waarden van K_d en herhaal iedere meting drie keer.
2. Bekijk zowel de hoek als de PWM-uitgang. Minder overshoot kan samengaan met een onrustiger stuursignaal. Verlaag K_d wanneer meetruis de motoruitgang sterk laat fluctueren.

3 Kies op basis van de resultaten een eindinstelling. Als PI beter presteert dan PID, is $K_d = 0$ een geldige uitkomst; motiveer dit met je metingen.
4 Test deze instelling bij een tweede haalbare doelhoek met dezelfde begincondities. Onderzoek of de afstelling ook daar bruikbaar is.

Verdieping — Setpointsprong tijdens de regeling

Pas in een eigen kopie het setpoint na een vast tijdstip eenmalig aan naar een tweede goedgekeurde hoek. Dit vraagt een codewijziging; het kan niet via een commando in de oorspronkelijke sketch. Registreer het veranderende setpoint mee. Laat de wijziging vooraf controleren en test een opgaande en neergaande sprong. Raak de bewegende arm niet aan om een verstoring te maken. Vergelijk de responsen en verklaar de invloed van de eenzijdige aandrijving.

Verwerking in Python

Het onderstaande voorbeeld leest de uit Serial Monitor opgeslagen CSV in. Gebruik een apart bestand per proef en neem de door de Arduino verstuurde kolomkop mee.

Meetbestand inlezen en controleren

```python
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('pid_meting.csv')
df['t_s'] = df['t_ms'] / 1000
dt = df['t_ms'].diff().dropna()
print('Meetinterval [ms]:', dt.describe())
if (dt <= 0).any():
    raise ValueError('Controleer op reset of onjuiste tijdstempels.')

fig, ax = plt.subplots(2, 1, sharex=True, figsize=(8, 6))
ax[0].plot(df['t_s'], df['angle_deg'], label='Gemeten hoek')
ax[0].plot(df['t_s'], df['setpoint_deg'], '--', label='Setpoint')
ax[0].set_ylabel('Hoek [graden]')
ax[0].legend()
ax[1].plot(df['t_s'], df['pwm'])
ax[1].set_ylabel('PWM [telwaarden]')
ax[1].set_xlabel('Tijd sinds startcommando [s]')
for a in ax:
    a.grid()
fig.tight_layout()
plt.show()
```

Gebruik de geregistreerde tijdstempels voor de tijdas. Het meet- en regelinterval hoort ongeveer 20 ms te zijn. De bibliotheek schaalt de I- en D-term met die ingestelde periode; afwijkingen daarvan zijn relevant voor de regeling. Vergelijk daarom ook de spreiding van de tijdsintervallen.

Beoordelingscriteria

Kies voor alle proeven dezelfde analysemethode. Neem $t_0 = 0$ bij het startcommando; bij de verdiepingsopdracht gebruik je het tijdstip van de setpointsprong. Een opstartrespons vanaf rust is niet hetzelfde experiment als een sprong vanuit een geregeld evenwicht.

- **Stationaire fout:** $e_{\mathrm{ss}} = r - \bar{\theta}$, met de gemiddelde hoek over een aantoonbaar stabiel eindinterval, bijvoorbeeld de laatste vijf seconden. Geef ook de standaarddeviatie en de lengte van dit interval. Bij blijvende drift of oscillatie rapporteer je dat geen stationaire toestand is bereikt.
- **Overshoot bij een opgaande respons:** $O = \max(0, \theta_{\max} - r)$ in graden. Eventueel normaliseer je als $100\%\, O / \mid r - \theta_0 \mid$, waarbij θ_0 de beginhoek is en $r \neq \theta_0$.
- **Insteltijd:** tijd vanaf t_0 totdat de hoek binnen een vooraf gekozen band rond het setpoint blijft, bijvoorbeeld $r \pm 2^\circ$, tot het einde van de registratie. Vereis daarna nog minimaal vijf seconden meettijd. Als dit niet gebeurt, rapporteer je “niet bereikt binnen de meetduur”.
- **Verzadiging:** geef aan of en hoelang de uitgang nabij nul of de ingestelde bovengrens blijft. Gebruik voor de tijdsduur de tijdstempels en noteer de gebruikte tolerantie.

Opdracht 5 — Vergelijken en verklaren

1. Controleer meetinterval, ontbrekende waarden en eventuele resets. Leg uit welke gegevens bruikbaar zijn en waarom.
2. Maak voor de gekozen P-, PI- en PID-instelling grafieken van hoek, setpoint en PWM. Gebruik dezelfde asschalen en lijn de proeven uit op t_0.

3 Maak een tabel met per instelling K_p, K_i, K_d, stationaire fout, overshoot en insteltijd. Rapporteer de spreiding over de drie herhalingen.
4 Verklaar de verschillen tussen P, PI en PID aan de hand van de drie regelbijdragen. Betrek de PWM-grafieken bij je verklaring.
5 Vergelijk de doelhoek en benodigde stationaire PWM met je open-loopmetingen. Bespreek waarom terugkoppeling kan corrigeren voor afwijkingen, maar geen onhaalbare hoek bereikbaar maakt.
6 Formuleer een conclusie: welke instelling voldoet het best aan jouw vooraf gekozen criteria? Bespreek de invloed van meetruis, timing, wrijving en uitgangsbegrenzing.

Afronding

1 Schakel de motorvoeding uit en wacht tot de propeller stilstaat.
2 Laat de gekozen afstelling en meetresultaten controleren door de practicumbegeleider.
3 Bewaar de gebruikte sketch, ruwe meetbestanden, instellingen en Python-analyse bij elkaar in je practicum-map.
4 Werk je labjournaal bij met de meetmethode, grafieken, vergelijkingstabel en onderbouwde conclusie.